ChatGPT

Technische Backend-Analyse von EVA AI (`SchBenedikt/nextcloud-ai`)

Executive Summary

Diese Analyse betrachtet den Stand von `main` sowie **sämtliche derzeit offenen Pull Requests** des Repositories zum **28. Oktober 2026**. `main` steht auf Commit `24e7982af575f2ef69d26a76912124e0cee47501` vom 13. Oktober 2026. Offen sind aktuell genau drei PRs: **#697**, **#665** und **#664**. PR **#697** ist der für die Architektur mit Abstand relevanteste: Er liegt direkt auf dem aktuellen `main`, ist offen, nicht als Draft markiert und laut GitHub mergebar. Die beiden anderen offenen PRs sind Dependabot-Upgrades für PHPUnit und Symfony YAML.

Gesamturteil: EVA hat bereits eine ungewöhnlich solide Backend-Basis für eine Nextcloud-App: klare Service-Trennung, user-spezifische Datenisolation, verschlüsselte Credentials und Chat-Historien, eine zentralisierte Tool-Policy, idempotente bzw. abbrechbare Indexierung, einen Embedding-Cache, Multi-User-Scheduling, API-Key-Scopes, Usage-Metriken sowie einen hybriden Retriever. Der größte technische Schwachpunkt ist jedoch inzwischen fundamental: **die semantische Suche ist keine echte Vector-DB-Suche**. Embeddings werden als JSON in einem SQL-`TEXT`-Feld gespeichert, aus der Datenbank nach PHP übertragen, dort per `json_decode()` deserialisiert und anschließend per PHP-Schleife mit Cosine Similarity verglichen. Bei mehr als 8.000 Chunks scannt EVA für die dense Candidate Selection sogar den **gesamten Benutzerindex in 2.000er-Seiten**. Das macht eine semantische Anfrage asymptotisch zu einem O(N·d)-Vollscan und wird ab mittleren Datenmengen zum dominanten Skalierungsproblem.

Die lexikalische Hälfte des Hybrid-Retrievals skaliert ebenfalls nur begrenzt: Für große Indizes erzeugt `ChunkMapper::filterChunksByTokens()` bis zu zwölf Bedingungen der Form `content LIKE '%token%'`. Es existiert in der Chunk-Tabelle nur ein Index auf `document_id`, nicht auf Inhalt oder Embedding. BM25 wird anschließend in PHP berechnet. Damit besitzt EVA zwar **logisch** einen guten Hybrid Retriever — BM25 + dense retrieval + RRF + Phrase Boost + Document Diversification — aber die beiden eigentlichen Retrieval-Primitive laufen weitgehend außerhalb spezialisierter Suchindizes.

Meine klare Empfehlung ist deshalb, eine `VectorStore`-Abstraktion einzuführen und als primäres Produktionsziel Qdrant zu implementieren. Qdrant passt besonders gut zur bestehenden Architektur: REST API, Self-Hosting, Managed/Hybrid/Private Cloud, Payload Filtering, dense und sparse Vectors, natives BM25, native RRF/Hybrid Queries und HNSW. Der bestehende EVA-RRF-Ansatz lässt sich dadurch nahezu semantisch erhalten, während der vollständige PHP-Cosine-Scan verschwindet. Qdrant unterstützt mittlerweile explizit die Kombination von Dense Retrieval und BM25 mit RRF im selben Query-Mechanismus. [1] Managed Qdrant verwendet dieselbe Open-Source-Engine wie Self-Hosting, und die Managed-Plattform bietet unter anderem Backups, horizontale Skalierung, Monitoring, Prometheus/OpenMetrics und Audit Logs. [2]

Weaviate ist die stärkste Alternative, insbesondere wenn EVA lexikalisches BM25 und Vector Search möglichst vollständig an eine Suchengine delegieren soll. Weaviate kombiniert BM25 und Vector Search nativ in Hybrid Search und bietet HNSW, Flat- und Dynamic-Indizes sowie Multi-Tenancy. [3] **Milvus** ist technisch sehr stark, aber für eine Nextcloud-App betrieblich schwerer: Standalone ist vertretbar,

Distributed bringt dagegen zusätzliche Komponenten und Kubernetes-nahe Komplexität mit sich und ist eher für sehr große Datenmengen interessant. [4] **Pinecone** ist eine attraktive Zero-Ops-Alternative, aber sie verschiebt personenbezogene Nextcloud-Inhalte in einen externen Managed Service und erzeugt stärkere Vendor- und Netzwerkabhängigkeit. [5]

Chroma Cloud sollte neu nicht mehr eingesetzt werden. Am 23. Oktober 2026 hat Chroma offiziell angekündigt, den bestehenden Cloud-Dienst abzuwickeln: Writes werden am **22. November 2026**, Reads am **22. Dezember 2026** abgeschaltet. Open-Source-Chroma bleibt bestehen. [6] Self-hosted Chroma bleibt damit technisch möglich, ist für EVA gegenüber Qdrant/Weaviate aber kein überzeugenderer Produktionspfad. Chroma selbst empfiehlt für Produktion eine servergestützte Konfiguration statt des lokalen Persistent Clients. [7]

`pgvector` **wäre architektonisch elegant, aber nicht als universelles EVA-Backend.** PostgreSQL mit pgvector bietet HNSW und IVFFlat sowie gefilterte ANN-Suche. [8] EVA ist aber eine Nextcloud-App und muss Installationen mit MariaDB/MySQL und anderen unterstützten Datenbankkonfigurationen berücksichtigen; Nextcloud 35 empfiehlt PostgreSQL oder MariaDB und unterstützt nicht ausschließlich PostgreSQL. [9] pgvector sollte deshalb höchstens ein optionaler Adapter sein, nicht der einzige neue Retrieval-Backend-Typ.

Cortex bewerte ich hier als das aktuelle Convex/Cortex-Memory-System. Es ist eher eine Memory-/Context-Schicht auf Convex als ein Drop-in-Ersatz für EVAs Dokument-Vektorindex. Der zusätzliche Plattform- und Datenmodellwechsel wäre deshalb im Verhältnis zum Nutzen hoch; für Agent Memory könnte es später interessant sein, für `eva_ai_chunks` nicht. Convex selbst bietet Vector Search als Plattformfunktion und berechnet Search Storage/Search Queries separat. [10]

Die zweite große Skalierungsbaustelle liegt außerhalb des Vector Search: `ChatStore` **speichert die komplette Chat-Historie eines Benutzers als ein AppData-JSON-Dokument**. Listen, Suchen und Mutationen laufen unter einem per-user Lock; die Inhaltssuche iteriert über die gespeicherten Messages, und Mutationen schreiben den aggregierten Store wieder zurück. Die Implementierung ist sorgfältig abgesichert und verschlüsselt neue Payloads mittels Nextcloud `ICrypto`, aber bei sehr großen Chat-Historien entsteht ein per-user Hot File mit Parse-/Decrypt-/Rewrite-Amplification und Lock-Contention. Eine mittelfristige Normalisierung in relationale `chat` / `message`-Tabellen wäre sinnvoller als der Versuch, diesen Store durch eine Vector DB zu ersetzen.

PR #697 ist insgesamt ein **positiver Backend-Schritt**. Neben dem UI-Umbau führt er Provider Profiles mit getrennten verschlüsselten Keys und explizitem `auth=none` für lokale OpenAI-kompatible Server ein und fügt einen günstigen `GET /api/index/status`-Pfad hinzu, der beim Polling bewusst auf Model Discovery, OCR-Probes, Chat Reads und Document Counts verzichtet. Der PR meldet lokal 723 PHPUnit-Tests mit 10.213 Assertions, 47 Frontend-Unit-Tests und 29 Browser-Tests; Produktions-Inference bei Hosted Providern wurde ausdrücklich nicht getestet.

Ein kritisches Delivery-Risiko liegt dagegen **außerhalb des Codes**: `main` ist derzeit nicht geschützt und hat laut GitHub keine erzwungenen Required Status Checks. Das sollte noch vor einer größeren Storage-Migration geändert werden.

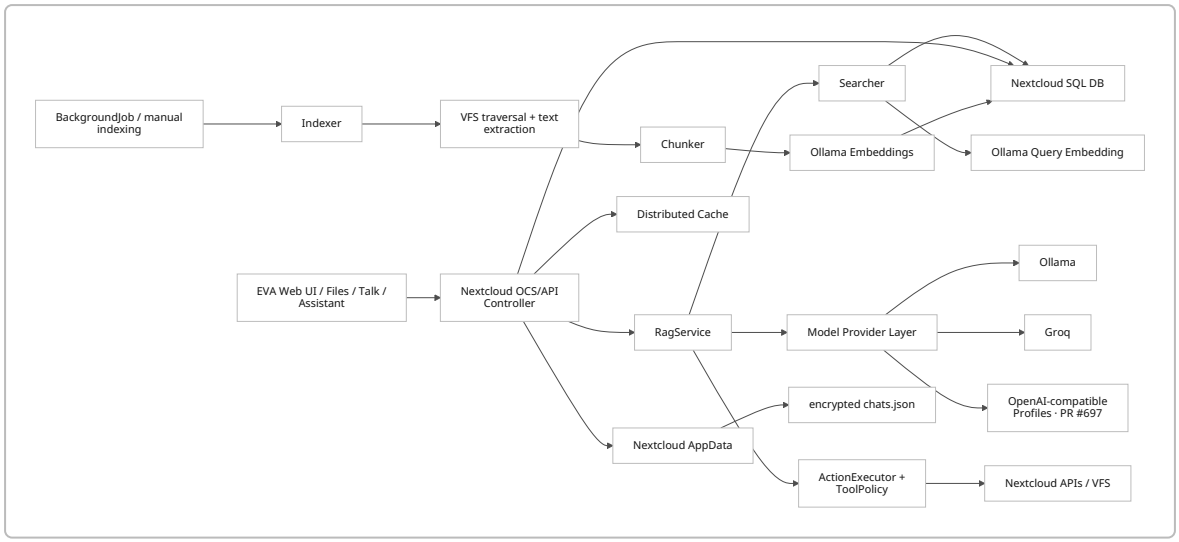
Prioritäten in Kurzform:

Priorität	Maßnahme	Nutzen	Aufwand
P0	`main` schützen, CI als Required Checks, DB-Matrix erweitern, SSRF-/Egress-Grenzen für Provider Profiles	sehr hoch	2–5 Tage
P0	reproduzierbares Retrieval-/Indexing-Benchmarking aufbauen	sehr hoch	3–5 Tage
P1	`VectorStore`-Abstraktion + Qdrant-Adapter + Generation-Metadaten	sehr hoch	5–8 Tage
P1	Dual Write + Backfill + Shadow Read	sehr hoch	6–10 Tage
P1	Dense Read auf Qdrant umschalten	extrem hoch	2–4 Tage
P2	BM25/RRF ebenfalls nach Qdrant oder Weaviate-artiges Hybrid Backend verlagern	hoch	4–8 Tage
P2	OpenTelemetry/Prometheus-artige Backend-Metriken und Tracing	hoch	4–7 Tage
P3	`ChatStore` in normalisierte SQL-Tabellen migrieren	mittel–hoch	7–12 Tage
P3	Indexing über persistente Outbox/Worker-Pipeline entkoppeln	hoch bei großen Instanzen	5–10 Tage

Unter der Annahme eines Entwicklers mit guter Kenntnis des Repositories ist eine **sichere Qdrant-Migration ohne Big-Bang in ungefähr 20–36 Engineering-Tagen** realistisch; der produktive Cutover sollte zusätzlich eine mehrtägige Shadow-/Soak-Phase enthalten. Das ist eine Aufwandsschätzung aus dem vorliegenden Codeumfang, keine Messung aus einer laufenden Instanz.

Ist-Architektur und Datenflüsse

EVA ist aktuell keine getrennte „AI Backend Platform“, sondern eine relativ eng in Nextcloud integrierte App. Die Architektur verwendet Nextclouds Datenbank, VFS/AppData, Cache-, Locking-, Background-Job-, Benutzer- und Crypto-Abstraktionen und bindet die eigentliche Inference über Ollama, Groq beziehungsweise mit PR #697 zusätzliche OpenAI-kompatible Endpunkte an. Die Repository-Dokumentation beschreibt `Indexer`, `Chunker`, `Ollama`, `Searcher`, `RagService`, `ActionExecutor`, `ToolPolicy` und die einzelnen Integrationen explizit als zentrale Services.



Der Indexierpfad ist im Repository bewusst bounded aufgebaut. `Indexer` verwendet einen Generator statt einen vollständigen Dateibaum in den RAM zu laden, besitzt per-user Locking, einen globalen Scheduler, Heartbeats und Abandoned-Run-Recovery. Der Standardwert des Embedding-Batches beträgt 24 und kann bis 200 konfiguriert werden. Extrahierte Inhalte werden auf zwei Millionen Zeichen begrenzt; dekomprimierte Archive werden ebenfalls begrenzt. Unveränderte Dateien können über gespeicherte Größe und `mtime` den teuren Content-/Embedding-Pfad überspringen.

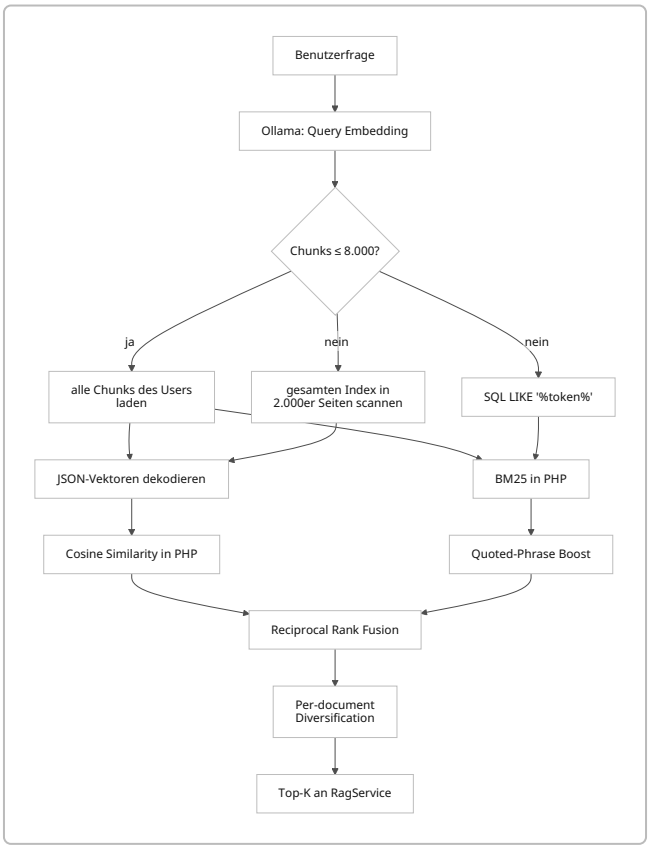
Das ist eine gute Ausgangslage. Besonders positiv ist, dass die Skalierungsprobleme **nicht** primär aus einem unbounded File Walker entstehen, sondern aus dem späteren Retrieval.

Persistenzschichten

Schicht	Aktuelle Daten	Bewertung
Nextcloud SQL	Dokumente, Chunks, Embeddings, Usage, API Keys, Agent-/weitere Metadaten	zuverlässig, aber Embeddings sind hier im falschen physischen Format
`eva_ai_chunks.embedding`	JSON-serialisierter Vektor in `TEXT`	zentraler technischer Engpass
Nextcloud AppData	Chat-Historien, weitere app-spezifische Dateien/Marker	passend für kleine Payloads; Chat-Aggregat skaliert schlecht
Distributed Cache	Embedding Cache, Health/ Greeting Cache, API-Key-Rate-Limit u. a.	sinnvoll; nicht als Source of Truth genutzt
Nextcloud VFS	Quelldateien, `KNOWLEDGE.md`, generierte Dateien	korrekte Autorität für Benutzerdateien
Externe AI-Endpunkte	Chat/Embeddings/Multimodal	bewusst austauschbar, Embeddings aber noch stärker an Ollama gekoppelt

Das initiale Schema speichert Dokument-Metadaten mit `user_id`, `file_id`, Pfad, MIME, Größe und Content Hash; Chunks enthalten `document_id`, `chunk_index`, `content`, `embedding` als `text` und `token_count`. Für Chunks wird lediglich `document_id` indiziert. Das `Chunk`-Entity serialisiert und deserialisiert den Vektor explizit mit JSON.

Aktueller Suchpfad



Bei einem kleinen Benutzerindex bis 8.000 Chunks lädt `Searcher` alle Chunks. Oberhalb dieses Werts werden lexical und dense Candidates separat gebildet. Der lexikalische Pfad verwendet bounded `LIKE`-Queries; der Dense-Pfad besucht dagegen für korrekte semantic-only Matches **jede Chunk-Seite des Benutzerindexes**, deserialisiert die Embeddings und hält nur die besten Cosine Scores. Genau diese Verbesserung gegenüber einem früheren Sampling-Ansatz schützt die Retrieval-Qualität, macht die Kosten aber proportional zur Indexgröße.

Der Retriever selbst ist konzeptionell gut. BM25 und Dense Ranking werden mit Reciprocal Rank Fusion kombiniert; quoted phrases erhalten einen exakten Boost und das Ergebnis wird anschließend auf mehrere Dokumente diversifiziert. Das sollte bei einer Migration **nicht weggeworfen**, sondern auf indizierte Primitive abgebildet werden. Qdrant unterstützt heute Dense + BM25 + RRF nativ; Weaviate kombiniert BM25 und Vector Search ebenfalls als native Hybrid Search. [11]

Model- und Provider-Pfad mit PR #697

PR #697 macht den Chat-Provider deutlich flexibler, verändert aber die Embedding-Seite noch nicht gleichermaßen. Provider Profiles können bis zu 20 Profile enthalten, haben IDs, URL, Modell und Auth-Modus; IDs dürfen `ollama` oder `groq` nicht überschreiben. Zulässig sind `api_key` und `none`.

URLs werden syntaktisch validiert und dürfen keine eingebetteten Credentials, Query-Parameter oder Fragmente enthalten.

Credentials liegen getrennt von den Provider-Profilen und werden per Nextcloud `ICrypto` verschlüsselt. Das gilt sowohl für Groq als auch für generische Provider-Felder und optional gespeicherte Nextcloud App Tokens.

`OpenAICompatible` implementiert unter anderem `/models`, `/chat/completions`, Image Generation und Audio/TTS. Bei `auth=none` wird bewusst kein Bearer Token übertragen. Bei von einem Image-Provider zurückgegebenen URLs akzeptiert der Code nur HTTPS und deaktiviert Redirects; Bilder werden außerdem größen- und formatseitig geprüft.

Wichtig für die Vector-DB-Planung: PR #697 beschreibt weiterhin explizit, dass **Document Indexing unabhängig vom Chat-Provider das Ollama-Embedding-Modell verwendet**. Deshalb sollte die Vector-DB-Arbeit mit einer separaten `EmbeddingProvider`-Abstraktion verbunden werden. Ein Chat-Provider-Profil und ein Embedding-Profil sind konzeptionell zwei verschiedene Dinge.

Detaillierte technische Befunde

Die folgenden Bewertungen sind nach Auswirkung priorisiert.

Befund	Schwere	Auswirkung	Empfohlene Richtung
PHP-Vollscan aller Embeddings bei großen Indizes	**kritisch**	Suchlatenz und PHP/DB-Last wachsen mit N×Dimension	ANN Vector Store
Embeddings als JSON-`TEXT`	**kritisch**	DB-Bloat, Transfer, JSON-CPU, kein ANN-Index	native Vector Storage
`LIKE '%token%'` als lexical prefilter	**hoch**	teure Scans, schlechte Skalierung	native BM25/FTS
Gesamter per-user Chat-Store in `chats.json`	**hoch**	Lock-/Rewrite-/Search-Amplification	normalisierte SQL-Tables
Embeddings weiterhin Ollama-zentriert	**hoch**	Inference-SPOF und eingeschränkte Providerwahl	`EmbeddingProvider` Interface
Custom Provider URLs als Outbound Request Surface	**hoch**	SSRF-/Egress-Risiko	private-range/DNS/ Egress Policy
Unprotected `main`	**hoch**	CI kann umgangen werden	Branch Protection
DB-Integrationstests primär SQLite	**mittel–hoch**	Portabilitätsregression möglich	PostgreSQL/MariaDB Matrix
Observability ohne Distributed Tracing	**mittel**	Bottleneck-Lokalisierung schwierig	OTel + Backend Metriken
Breite API-/Controller-Fläche	**mittel**	steigende Kopplung/Review-Komplexität	Controller nach Domänen teilen

Befund	Schwere	Auswirkung	Empfohlene Richtung
Rate Limiting stark cacheabhängig	**mittel**	Cache-Störung kann externe API blockieren	explizites Resilience Design
Dual Storage bei Vector-DB-Einführung	**neu, hoch**	Konsistenzrisiko	Transactional Outbox

Dense Retrieval ist der zentrale Bottleneck. `Searcher::loadCandidates()` scannt bei großen Indizes den gesamten Index in `DENSE_SCAN_PAGE = 2000`-Seiten und berechnet Cosine Similarity selbst. Ein Benutzer mit einer Million Chunks verursacht damit pro semantischer Anfrage Größenordnungen mehr DB-I/O und PHP-Arbeit als bei einer ANN-Abfrage, deren Index gerade dazu dient, nicht alle Vektoren zu besuchen.

Als Größenordnung: Eine Million 768-dimensionale `float32`-Vektoren enthalten bereits etwa **3,07 GB rohe Vektordaten**, bevor ANN-Strukturen, IDs, Payloads oder Replikate hinzukommen. Die aktuelle JSON-Repräsentation ist deutlich größer als die rohe binäre float32-Repräsentation. Diese Größen sind einfache Speicherberechnungen; der reale DB-Footprint muss auf EVA selbst gemessen werden.

Lexical Retrieval ist der zweite Engpass. `filterChunksByTokens()` generiert `LIKE`-Prädikate mit führendem `%` für bis zu zwölf Query-Tokens. Die anschließende BM25-Berechnung erfolgt außerhalb der Datenbank. Die Vector-DB-Migration sollte daher nicht bei „Cosine wird schneller“ stehenbleiben. Das Endziel sollte ein echter **Hybrid Search Index** sein.

Chat Storage ist eine separate Skalierungsachse. `ChatStore` serialisiert den Zugriff mit einem User Lock und hält bis zu 1.000 Messages je Conversation. Die Listensuche durchsucht Messages in PHP; Mutationen führen wieder zu einem Schreiben des aggregierten Benutzerstores. Das ist heute kein Grund, die Chat-Historie in Qdrant zu verschieben. Die richtige Trennung ist:

`SQL = kanonische Chats/Messages`
`Vector Store = optionaler semantischer Chat-Memory-Index`

Ein relationales Modell ermöglicht Pagination, Indizes auf `updated_at`, einzelne Message Writes und saubere Retention, ohne die komplette History jedes Mal zu entschlüsseln und neu zu schreiben.

Die Verschlüsselung der Chat-Historie ist dagegen positiv. Der Store verschlüsselt neue Payloads mit `ICrypto`; bestehende Plaintext-Stores können zur Migration gelesen werden, und ein verschlüsselter Store fällt bei fehlender Crypto-Fähigkeit nicht einfach auf Klartext zurück.

Indexing ist deutlich besser gebaut als Retrieval. Generator-basierter Walk, Batching, per-user Locking, globales Scheduling, Heartbeats, Cancellation und Fast Paths zeigen, dass mehrere klassische Indexer-Probleme bereits adressiert wurden. Der nächste Schritt sollte deshalb keine komplette Indexer-Neuschreibung sein, sondern die Einführung einer sauberen Persistenzschnittstelle hinter dessen Chunk-/Embedding-Pfad.

Embedding Cache ist wiederverwendbar. EVA besitzt bereits einen 30-Tage-Content-addressed Distributed Cache. Keys sind user-isoliert und berücksichtigen Endpoint, Embedding-Modell und Schema; Dimensionen werden validiert. Ein Cache-Ausfall lässt Indexing nicht scheitern. Dieser Cache ist bei einer Reindexierung wertvoll: vorhandene kompatible Embeddings müssen nicht unnötig neu berechnet werden.

Ein neues Vector Backend erzeugt sonst einen Distributed-Transaction-Bug. Heute ist der SQL-Store im Wesentlichen eine Persistenzdomäne. Sobald Indexer gleichzeitig SQL und Qdrant schreibt, kann „SQL erfolgreich, Qdrant fehlgeschlagen“ beziehungsweise der umgekehrte Fall auftreten. Deshalb sollte die Zielarchitektur nicht auf naivem Best-Effort-Dual-Write enden. Eine kleine **transactional outbox** in der Nextcloud-Datenbank ist wesentlich robuster:

```
SQL transaction:
    upsert document/chunk metadata
    insert vector_outbox event
COMMIT

Vector worker:
    claim event
    idempotent Qdrant upsert/delete
    mark event delivered
```

Qdrant kann Points idempotent über stabile IDs upserten; die genaue Client-Implementierung kann über dessen REST API erfolgen. Qdrant dokumentiert Bulk Upserts und laufende Inserts/Updates/Deletes als normalen Betriebsmodus. [12]

Provider Profiles erweitern die SSRF-Vertrauensgrenze. PR #697 erlaubt benutzerdefinierte HTTP(S)-URLs und explizit unauthentifizierte lokale Server. Die aktuelle Validierung verhindert URL-Credentials, Query und Fragment, nicht aber grundsätzlich private, Loopback- oder Link-Local-Ziele. Das ist für LM Studio/vLLM/Ollama-artige lokale Dienste funktional erforderlich, gleichzeitig aber eine Egress-Sicherheitsgrenze. Meine Empfehlung ist daher nicht „private URLs generell verbieten“, sondern:

Admin Policy → Provider URL Validator → DNS-Auflösung → IP-Klassifizierung → Egress Request

Standardmäßig sollten Link-Local/Metadata-Adressen blockiert werden; Loopback/private Netzbereiche sollten nur nach expliziter Admin-Policy erlaubt sein. Redirects müssen erneut geprüft werden. DNS-Rebinding-Szenarien sollten ebenfalls durch Resolve-and-Validate beziehungsweise Netzwerk-Egress-Regeln adressiert werden. Das ist eine aus dem vorliegenden Request-Pfad abgeleitete Sicherheitsmaßnahme, kein behaupteter bestätigter Exploit.

Die externe API ist ansonsten gut begrenzt. API Keys werden als zufällige `eva_sk_...`-Secrets erzeugt; in SQL liegt nur SHA-256 plus Prefix. Es gibt Scopes `read`, `write`, `admin`, Ablaufzeiten und optionale exakte IP-Allowlisting. Externe Chats erhalten ausdrücklich `allowActions: false`; Dokumentzugriffe prüfen User Ownership. Pro Key gelten im aktuellen Code Minute-Budgets von 100 Read-, 30 Write- beziehungsweise 10 Admin-Requests, umgesetzt über Nextclouds Locking Cache.

Observability ist vorhanden, aber auf Produktmetriken fokussiert. `UsageMetrics` speichert keine Prompts, Antworten oder Tool-Argumente, sondern Provider/Modell, Tokenzahlen, Dauer und bei langsamen Tools aggregierbare Timingdaten. PR #697 ergänzt den schlanken Indexstatus und hält Provider-Key-Werte selbst aus Statusantworten heraus. Was fehlt, sind insbesondere Retrieval-Span-Timings, Queue Lag, Embedding-Latency-Histogramme, Search p95/p99, DB Read Volume, Vector-DB-Saturation und durchgängige Trace IDs.

PR #697 selbst bewerte ich als merge-würdig nach den üblichen Review-Gates. Backend-seitig sind besonders der günstige Index-Status, die Provider-Profile, getrennte verschlüsselte Credentials und das

explizite `auth=none` relevant. Der PR berichtet umfassende Tests, weist aber selbst darauf hin, dass reale Hosted-Provider-Inference nicht ausgeübt wurde. Genau diese Lücke sollte vor breiter Provider-Freigabe durch Mock-Contract-Tests plus wenigstens opt-in Nightly Live Tests geschlossen werden.

Die übrigen offenen PRs sind Wartungs-PRs. #665 hebt PHPUnit von 11.5.56 auf 13.3.5 an. #664 hebt Symfony YAML von 6.4.45 auf 8.1.8; die mitgelieferten Upstream-Release-Notes nennen unter anderem Sicherheits-Härtungen gegen katastrophales Regex-Backtracking und unbounded Collection-Alias-Auflösung. Beide sind Major-Upgrades und sollten nicht als triviale Dependabot-Merges behandelt werden: die bestehende PHP-/Nextcloud-Testmatrix muss vollständig grün sein; für Symfony sollte zusätzlich geprüft werden, ob ein sicherer Patch innerhalb der kompatiblen Release-Linie vorzuziehen ist.

Vektordatenbanken und Zielarchitektur

Die Wahl sollte nicht nach einem synthetischen „welche DB ist die schnellste?“ erfolgen. Ohne identischen EVA-Datensatz, identische Embeddings, Filterselektivität, Recall-Ziel und Hardware wäre ein solcher Vergleich methodisch wertlos. Entscheidend sind zunächst **Architektur-Fit, Betriebsmodell, Privacy, Hybrid Retrieval und Migrationskomplexität**.

Kandidat	Hybrid Search	Self-host	Managed	PHP/EVA-Integration	Betriebsaufwand	EVA-Fit
Qdrant	Dense + Sparse + natives BM25 + RRF	sehr gut	ja	REST sehr gut	niedrig–mittel	**5/5**
Weaviate	natives BM25 + Vector Fusion	sehr gut	ja	HTTP gut	mittel	**4.5/5**
Milvus	Vector/ Hybrid/ Metadata	ja	Zilliz	REST verfügbar	mittel–sehr hoch	**3.5/5**
Pinecone	Dense/ Sparse/Full-Text	nein klassisch; BYOC/ Managed	ja	API einfach	sehr niedrig	**3.5/5**
pgvector	Vector + PostgreSQL FTS kombinierbar	Bestandteil PostgreSQL	über PG-Provider	technisch sehr gut	niedrig	**3/5 universell; 4.5/5 PG-only**
Chroma OSS	Dense/ Sparse/Full-Text	ja	bestehende Cloud endet	HTTP möglich	niedrig–mittel	**2.5/5**
Cortex/ Convex	Memory/ Search-Plattform	anderes Modell	Convex	hoher Modellwechsel	mittel	**1.5/5 als Chunk Store**

Qdrant unterstützt die für EVA sehr relevante Kombination aus Dense Retrieval und BM25 sowie RRF direkt im Query API. [1] Qdrant Cloud bietet Self-host-kompatible APIs, Managed, Hybrid und Private Cloud; die Free Tier besteht derzeit aus einem einzelnen Node mit 0,5 vCPU, 1 GB RAM und 4 GB Disk. [13]

Weaviate besitzt getrennte Vector- und Inverted-Indizes; Hybrid Search führt BM25 und Vector Search parallel aus und fusioniert die Ergebnisse. HNSW, Flat, Dynamic und disk-orientierte Optionen stehen zur Verfügung; der Dynamic Index kann bei kleinen Tenants mit Flat beginnen und später auf HNSW wechseln. [3] Das ist insbesondere bei vielen kleinen Nextcloud-Benutzern attraktiv.

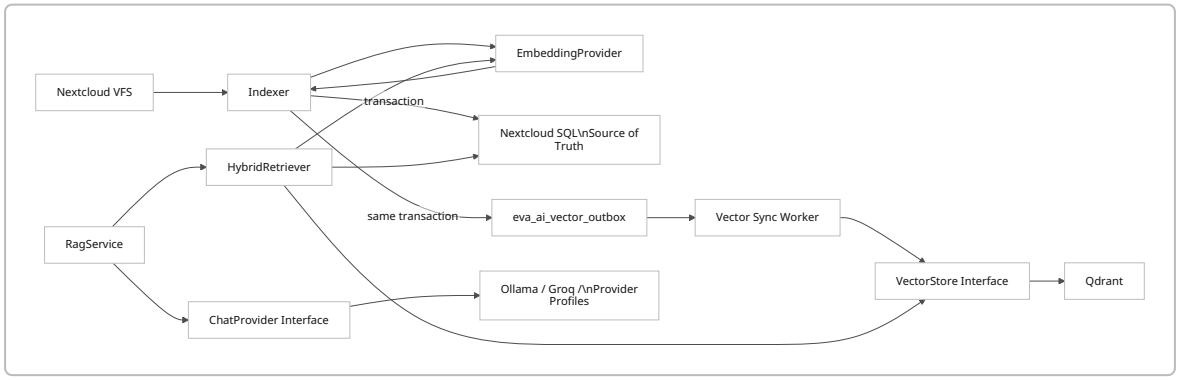
Milvus ist die stärkste Scale-out-Option im Vergleich. Die offizielle Architektur trennt Access-, Coordination- und Data-Plane-Komponenten; Milvus bietet Lite, Standalone und Distributed. Die Dokumentation ordnet Standalone bis in den Bereich großer Einzelserver-Deployments und Distributed bis hin zu sehr großen verteilten Datensätzen ein. [4] Für eine typische EVA-Installation wäre diese Flexibilität jedoch überwiegend ungenutzte Komplexität.

Pinecone reduziert Operations massiv und unterstützt Dense, Sparse und Full-Text Indizes. Das aktuelle Pricing hat einen kostenlosen Starter, einen Builder für 20 US-Dollar monatlich, Standard mit 50 US-Dollar Mindestverbrauch und Enterprise mit 500 US-Dollar Mindestverbrauch. [14] Für lokale/private Nextcloud-Deployments bleibt der externe Datentransfer aber ein wesentliches Architektur- und Datenschutzkriterium.

pgvector unterstützt exakte Suche sowie ANN mit HNSW und IVFFlat. HNSW bietet laut Projekt einen besseren Speed/Recall-Trade-off als IVFFlat, benötigt aber mehr Speicher und längere Build-Zeit; gefilterte ANN-Abfragen und iterative Scans werden unterstützt. [8] Das wäre für eine reine PostgreSQL-Anwendung wahrscheinlich meine Minimalismus-Empfehlung. EVA kann sich als Nextcloud-App aber nicht darauf verlassen, dass PostgreSQL vorhanden ist. [9]

Chroma OSS besitzt inzwischen Dense-, Sparse-, Hybrid-, Full-Text- und Metadata Search. [15] Trotzdem würde ich für ein neues EVA-Produktionsbackend aktuell Qdrant/Weaviate bevorzugen, weil der bisherige Managed Chroma Cloud Service bereits konkret eingestellt wird. Open-Source-Chroma bleibt zwar weitergeführt, der Cloud-Lifecycle ist aber ein zusätzlicher strategischer Risikofaktor. [6]

Empfohlene Zielarchitektur



Dabei sollte die interne API ungefähr folgende semantische Grenze haben:

```
interface VectorStore {
    public function upsert(array $chunks, VectorGeneration $generation):
void;

    public function deleteDocument(
        string $userId,
        int $documentId,
        VectorGeneration $generation
    ): void;

    public function search(
        string $userId,
        array $queryVector,
        int $topK,
        SearchFilters $filters,
        VectorGeneration $generation
    ): array;

    public function health(): VectorStoreHealth;
}
```

Die wichtige Designentscheidung ist **Vector Generation statt „ein globales Embedding-Modell“**. Qdrant Collections beziehungsweise Vector-Konfigurationen benötigen eine definierte Dimension; serverseitiges Inference schlägt beispielsweise fehl, wenn die Modell-Dimension nicht zur Collection passt. [16] Das wird für EVA besonders relevant, sobald verschiedene Benutzer unterschiedliche Embedding-Modelle verwenden dürfen.

Ich würde deshalb relational ergänzen:

```
eva_ai_vector_generations
──────────────────────────────────────
id
user_id
backend
collection
embedding_provider
provider_profile_id nullable
embedding_model
embedding_dimension
distance_metric
chunk_size
chunk_overlap
schema_version
status                 building|active|retired|failed
created_at
activated_at
```

und an Chunks:

```
vector_generation_id
vector_state           pending|synced|failed
content_hash
```

Die physische Qdrant-Repräsentation wäre beispielsweise:

```
{
  "id": 4815162342,
  "vector": {
    "dense": ["... float vector ..."]
  },
  "payload": {
    "user_id": "USER_NAMESPACE",
    "generation_id": "GENERATION_ID",
    "document_id": 927,
    "file_id": 1337,
    "chunk_index": 18,
    "source": "file",
    "path": "Documents/Projekt/plan.md",
    "content_hash": "..."
  }
}
```

`user_id`, `generation_id`, `document_id` und `source` sollten als filterbare Payload-Felder indiziert werden. Qdrant verwendet Payload-Indizes zur effizienten Filterung und kann Filter in die Vektorsuche integrieren. [12]

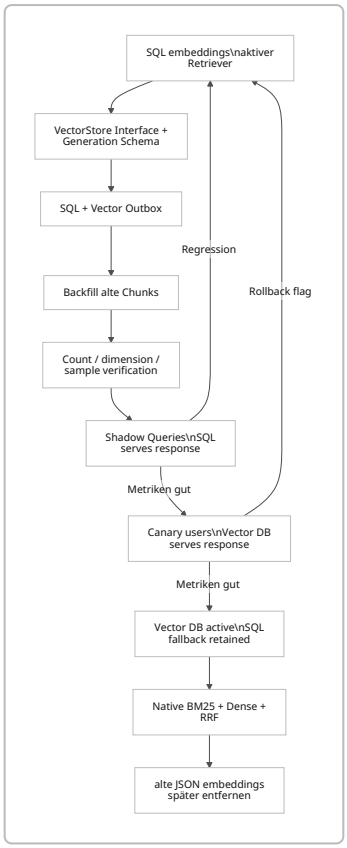
Phase eins sollte nur den Dense Vector externalisieren und den Chunk-Text weiterhin aus SQL hydrieren. Das minimiert Datenkopien und Migrationsrisiko.

Phase zwei kann den `content` zusätzlich als Qdrant-Text-Payload replizieren und einen Full-Text/BM25-Index verwenden. Dann kann der heutige PHP-RRF-Pfad komplett durch eine native Query aus Dense + BM25 + RRF ersetzt werden. Qdrant dokumentiert genau diesen Query-Typ. [1] Dadurch verschwindet auch das `LIKE '%token%'`-Problem.

Die SQL-Datenbank bleibt trotzdem **Source of Truth**. Qdrant ist ein rekonstruierbarer Retrieval-Index. Das vereinfacht Disaster Recovery, DSGVO-Löschung und Rollback erheblich.

Migrations- und Roadmapplan

Die Migration sollte niemals „alte Tabelle löschen, Qdrant aufbauen, hoffen“ sein. Sie lässt sich ohne längere Downtime als Generationenwechsel durchführen.



Backfill ohne unnötige Re-Embeddings. Für existierende Rows kann EVA vorhandene JSON-Vektoren direkt nach Qdrant übertragen, **sofern** Modell und Dimension zweifelsfrei der aktiven Generation zugeordnet werden können. Weil das aktuelle Chunk-Schema selbst keine Modell-ID je Embedding speichert, darf diese Zuordnung nicht geraten werden. In Zweifelsfällen muss neu eingebettet werden. Der existierende Embedding Cache kann kompatible Vektoren wiederverwenden und berücksichtigt bereits Modell, Endpoint und Dimension.

Reindexing sollte dokumentweise und checkpoint-basiert erfolgen:

```
generation = BUILDING
cursor = last_chunk_id

while work remains:
    read bounded batch
    obtain/validate embedding
    upsert stable point IDs
    persist cursor
    heartbeat

verify:
    SQL expected chunks == vector-store points
    dimension valid
    random cosine comparison
    tenant filters valid
```

```
generation = ACTIVE
previous_generation = RETIRED
```

Bei einem Embedding-Modellwechsel wird **nicht** die aktive Collection in place umgebaut. EVA erzeugt eine neue Generation, indexiert parallel, misst Shadow Quality und schaltet anschließend den `active_generation_id` um. Das löst gleichzeitig das Problem unterschiedlicher Vektordimensionen.

Rollback-Regel: JSON-Embeddings mindestens einen vollständigen Release-Zyklus nach dem Vector-DB-Cutover behalten. Solange SQL weitergeschrieben wird oder rekonstruierbare Embeddings enthält, ist Rollback nur ein Feature-Flag:

```
retrieval_backend = sql_legacy | vector_shadow | vector_primary
```

Erst wenn Vector Search mehrere Wochen stabil ist und Restore-/Reindex-Tests erfolgreich sind, sollte eine destruktive Migration das alte `embedding TEXT` entfernen oder nicht mehr befüllen.

Roadmap

Zeitraum	Milestone	Tasks	Exit-Kriterium	Rollback
Woche 0–1	**Safety baseline**	Branch Protection, CI Required Checks, SSRF Policy, Benchmark Corpus	reproduzierbare Baseline	keine Produktänderung
Woche 1–2	**Storage abstraction**	`VectorStore`, Generation Schema, Feature Flags	Legacy Backend unverändert grün	Interface deaktivieren
Woche 2–3	**Qdrant integration**	Connector, auth/TLS, payload indexes, health checks	CRUD + tenant tests grün	Qdrant abschalten
Woche 3–4	**Reliable writes**	Outbox, idempotente Upserts/Deletes, retry/dead-letter	Kill/retry ohne Divergenz	Legacy SQL bleibt aktiv
Woche 4–5	**Backfill**	resumable migration, cache reuse, dimension verification	100 % erwartete Points	Collection löschen
Woche 5–6	**Shadow retrieval**	beide Retriever ausführen, Ranking vergleichen	Qualität ≥ Baseline; p95-Ziel erfüllt	SQL beantwortet weiter
Woche 6–7	**Canary cutover**	5–10 % Nutzer/ Instanzen, Fallback	keine Leakage/ Fehlerrate	Flag zurück
Woche 7–8	**Primary vector read**	Qdrant default, SQL fallback	stabiler Soak	Flag zurück

Zeitraum	Milestone	Tasks	Exit-Kriterium	Rollback
Woche 8–10	**Native hybrid**	Qdrant BM25 + Dense + RRF, Phrase/Scope Tests	Qualitätsbenchmark bestanden	Dense Qdrant + alter BM25
Woche 10–12	**Hardening**	ChatStore SQL-Migration, OTel, DR, load tests	SLO + Restore Drill	separate Feature Flags

Geschätzter Engineering-Aufwand für den Vector-Pfad

Arbeit	Schätzung
Abstraktion + Schema + Feature Flags	3–5 Personentage
Qdrant Client + Auth + Filter + CRUD	3–5
Transactional Outbox + Worker	3–5
Backfill/Reindex + Checkpointing	3–5
Shadow Comparator + Telemetrie	3–5
Hybrid/BM25/RRF-Portierung	4–8
Cutover/Runbooks/DR	2–3
Gesamt	**20–36 Personentage**

Die größte fachliche Gefahr ist nicht der Connector, sondern die **Retrieval-Regressionsgefahr**. Der jetzige `Searcher` enthält inzwischen mehrere Qualitätsmechanismen — filename/path lexical context, quoted phrase boost, RRF und Dokument-Diversifizierung. Eine neue Vector DB muss gegen dieses Verhalten getestet werden, statt nur schnellere Cosine-Ergebnisse zu liefern.

CI/CD, Observability und Security

Das bestehende CI ist schon umfangreicher als bei vielen vergleichbaren Apps. `tests.yml` testet PHP 8.2, 8.3 und 8.4, erzeugt Coverage, führt einen Groq-Contract gegen Nextcloud-Interfaces aus, baut und testet das Frontend, verifiziert committed Bundles, führt Browser-Tests, Migrationstests und Nextcloud-App-Lifecycle-Tests gegen mehrere Nextcloud-Versionen durch und lintet PHP. Die Quality-Pipeline führt Composer- und npm-Audits aus; ESLint ist allerdings explizit nur advisory und darf fehlschlagen.

Das größte CI/CD-Problem ist deshalb nicht fehlendes Testing, sondern **fehlende Enforcement auf** `main`. GitHub meldet den Branch als unprotected und Required Checks als nicht erzwungen.

Empfohlene Merge Policy:

```
main
 ├─ require pull request
 ├─ require 1 review
 ├─ dismiss stale approvals
 ├─ require CODEOWNERS for security/storage paths
```

```
├─ require signed/verified commits where practical
└─ required checks
     ├─ phpunit
     ├─ migration-regression
     ├─ nextcloud-app
     ├─ frontend
     ├─ dependency-audit
     ├─ php-static-analysis
     ├─ DB portability
     └─ vector-integration
```

Die Datenbankmatrix sollte erweitert werden. Der derzeitige Nextcloud-Lifecycle-Job installiert Nextcloud mit SQLite. Nextcloud selbst empfiehlt PostgreSQL oder MariaDB für reguläre Installationen und bezeichnet SQLite nur für Testing beziehungsweise minimale Instanzen als geeignet. [9] Deshalb sollten mindestens PostgreSQL und MariaDB echte Migration-/Mapper-/API-Integrationstests erhalten.

Zusätzlich empfehle ich PHPStan auf hohem Level oder Psalm, CodeQL beziehungsweise vergleichbares SAST, Secret Scanning, Dependency Review, ein SBOM pro Release und SHA-gepinnte GitHub Actions. Diese Punkte sind neue Empfehlungen; die vorhandenen Workflows zeigen aktuell Dependency Audits und Syntax-/Unit-/Integrationstests, aber keine entsprechende erzwungene statische Analyse oder Supply-Chain-Gates.

Observability-Zielmodell

Die aktuelle `UsageMetrics`-Schicht sollte bestehen bleiben, weil sie bewusst ohne Prompt-/Antwortinhalt misst. Daneben braucht EVA echte technische Telemetrie.

Metrik/Span	Warum
`rag.request.duration` p50/p95/p99	Gesamt-RAG-Latenz
`embedding.query.duration`	lokales Modell vs. Provider erkennen
`retrieval.vector.duration`	Vector DB isolieren
`retrieval.lexical.duration`	BM25 isolieren
`retrieval.hydrate.duration`	SQL-Chunk-Load
`retrieval.candidates`	Search-Tuning
`retrieval.backend`	Legacy/Qdrant/Shadow
`index.chunks_per_second`	Indexing Throughput
`index.embedding_cache_hit_ratio`	Reindex-Kosten
`index.queue_age_seconds`	Backpressure
`vector.outbox.pending`	Konsistenzstatus
`vector.outbox.oldest_age`	Sync-Störung
`vector.query.errors`	Service Reliability

Metrik/Span	Warum
`chat_store.lock_wait_ms`	heutiger Hot-File-Effekt
`provider.duration/error_rate`	externe Abhängigkeiten
`tool.duration/error_rate`	Nextcloud Integration
`db.query.duration/read_rows`	DB Bottleneck

Für Qdrant existieren bereits OpenMetrics/Prometheus-Endpunkte, Health-Endpunkte, Grafana-Referenzdashboards und Audit Logging in den entsprechenden Betriebsvarianten. [12] Diese sollten in ein EVA-Dashboard mit Nextcloud-/PHP-FPM-/DB-Metriken zusammengeführt werden.

Security-Hardening

Die stärksten vorhandenen Kontrollen sollten beibehalten werden: getrennte und verschlüsselte Provider Secrets, keine Rückgabe entschlüsselter Keys im Status, hash-only API Keys, user-scoped Document API, `allowActions=false` für den externen Chat-Pfad und zentrale Tool Policy.

Vor einer Vector-DB-Einführung würde ich folgende Security-Invarianten als Tests festschreiben:

Invariante	Test
Benutzer A kann niemals Vektoren von B erhalten	adversarial cross-tenant query suite
Jede Search Query enthält serverseitig `user_id`	Connector contract test
Client darf Tenantfilter nicht überschreiben	request-fuzzing
User delete entfernt SQL + Vector Index + Cache	GDPR integration test
Backup-Retention ist dokumentiert	restore/delete drill
Qdrant nicht öffentlich erreichbar	deployment policy
Service Key nur serverseitig	secret scanning
TLS auf Remote Vector Store	integration check
Embedding Generation muss Dimension matchen	reject-on-write test
keine neue Generation wird vor Verification aktiv	state-machine test

Bei Qdrant Self-host sollte der Port nur im internen Netz erreichbar sein; bei Cloud sollte eine private Verbindung beziehungsweise IP-Policy erwogen werden. Managed Qdrant bietet unter anderem TLS, verschlüsselte Volumes und private Netzwerkoptionen abhängig vom Tarif. [2]

Die **Provider-URL-Policy** verdient hohe Priorität. Besonders zu testen sind `127.0.0.1`, `::1`, RFC1918-Ziele, Link-Local, Cloud-Metadata-Adressen, DNS-Rebinding, IPv4-in-IPv6, Redirects und ungewöhnliche Hostname-Darstellungen. Dabei sollte ein Admin lokale Provider gezielt freischalten können, statt legitime Self-hosting-Szenarien zu zerstören.

PR #697 hebt mehrere Frontend-Abhängigkeiten auf gepatchte Versionen und meldet für den Produktions-NPM-Audit null Vulnerabilities. Das ist positiv, ersetzt aber keine kontinuierliche Dependency Policy.

Benchmarks, Kosten und Grenzen

Vor dem Cutover sollte ein **EVA-spezifischer Benchmark** entstehen. Vendor-Benchmarks sind für die Vorauswahl nützlich, aber nicht geeignet, Qdrant, Weaviate, Milvus und Pinecone fair gegeneinander zu ranken, wenn Datenverteilung, Filter, Hardware und Recall-Ziel unterschiedlich sind.

Benchmark-Matrix

Scale	Chunks	Beispiel-Vektordimension	Rohdaten nur Vektoren
Small	100.000	768	~307 MB
Medium	1.000.000	768	~3,07 GB
Large	10.000.000	1.024	~40,96 GB

Die tatsächliche Storage-Anforderung liegt darüber, weil IDs, Payloads, Text, ANN-Indizes, WAL und gegebenenfalls Replikate hinzukommen.

Der Benchmark sollte vier Query-Klassen enthalten: rein semantische Fragen ohne gemeinsames Vokabular, exakte Begriffe/IDs/Dateinamen, gemischte natürliche Fragen und bewusst adversariale Scope-/Tenant-Anfragen. Zusätzlich sollte es Queries mit quoted phrases, mehreren relevanten Dokumenten und sehr ähnlichen Nachbar-Chunks geben, weil genau dafür der bestehende Retriever Sonderlogik besitzt.

Zu messende Qualitätsmetriken

`Recall@5/10/20`, `Precision@k`, `MRR@10` und `nDCG@10` sollten mit einem manuell oder synthetisch annotierten Query-Relevance-Set erfasst werden. Zusätzlich empfehle ich `source_document_recall`, `duplicate_chunk_rate`, `distinct_documents@k` und einen harten `cross_tenant_leakage = 0`-Test.

Als Vergleichsbaseline sind mindestens diese Systeme nötig:

```
A: heutiger EVA Searcher
B: exakter brute-force cosine baseline
C: Qdrant dense ANN
D: Qdrant dense + heutiger EVA lexical/RRF
E: Qdrant dense + native BM25 + RRF
F: optional Weaviate Hybrid
```

Dadurch wird getrennt sichtbar, ob Qualitätsänderungen vom ANN-Approximationseffekt oder von der Migration des BM25/RRF stammen.

Performance-Metriken

Gemessen werden sollten p50/p95/p99 für Query Embedding, Dense Retrieval, Lexical Retrieval, Fusion, SQL Hydration und komplettes Retrieval; zusätzlich CPU, PHP-RSS, DB-Zeilen/Bytes, Netzwerkvolumen, Vector-DB-RAM/Disk, Ingest Chunks/s und Reindex-Dauer. Bei Lasttests sollten 1, 5, 20 und 50 parallele Benutzer sowie Cold/Warm Cache unterschieden werden.

Ein vernünftiger erster Acceptance Gate wäre:

Kriterium	Ziel
Cross-user leakage	**0**
ANN Recall@10 gegen Exact Baseline	≥ 95 %
nDCG@10 gegen heutiges EVA	keine Regression > 2 %
Retrieval p95 bei 1 Mio. Chunks, exkl. LLM	< 200 ms als Startziel
Retrieval p99	< 500 ms als Startziel
Outbox nach Normalbetrieb	praktisch leer
verlorene Upserts nach Worker Kill	0
doppelte Upserts mit falschem Ergebnis	0
Indexierung nach Abbruch fortsetzbar	100 %
Rollback ohne Reindex	ja, während Migrationsfenster

Die konkreten Latenz-SLOs sollten nach dem ersten Hardware-Baseline-Test angepasst werden; sie sind Zielvorgaben, keine behaupteten aktuellen EVA-Werte.

Vector-DB-Kosten

Die folgende Einstufung trennt offizielle Listenpreise von groben Infrastrukturannahmen. Self-host-Kosten hängen stark vom vorhandenen Server ab; insbesondere Ollama und Vector DB auf demselben kleinen Host können um RAM/CPU konkurrieren.

Backend	Klein	Mittel	Groß/HA	Kostenurteil
Qdrant OSS	Software $0; ggf. vorhandener Host	zusätzliche VM typ. niedrige bis mittlere zweistellige bis niedrige dreistellige €-Kosten	mehrere Nodes: mehrere hundert € aufwärts	**niedrig–mittel**
Qdrant Cloud	Free: 0,5 vCPU/ 1 GB/4 GB	usage/resource-based	Multi-node/HA usage-based	**niedrig–hoch nach Ressourcen**
Weaviate Cloud	Free bis 100k Objects	Flex ab $45/Monat	Premium ab höherem Commit; HA-Tiers	**mittel**

Backend	Klein	Mittel	Groß/HA	Kostenurteil
Milvus OSS	Standalone, Software $0	größerer Standalone-Host	Distributed + K8s deutlich aufwendiger	**mittel–hoch ops**
Zilliz Cloud	Free verfügbar	etwa $20–65/Mio. 768d-Vektoren/Monat je Dedicated-Profil als Orientierung	Tiered/Capacity/ Performance variieren	**mittel**
Pinecone	Free bzw. Builder $20	Standard mindestens $50 + Usage	Enterprise mindestens $500 + Usage	**mittel–hoch**
pgvector	Extension $0	vorhandenes PostgreSQL skaliert mit	PostgreSQL-HA-Kosten	**sehr niedrig bei bestehendem PG**
Chroma OSS	Software $0	eigener Server	eigener Distributed-Betrieb	**niedrig–mittel**, aber strategisches Risiko
Chroma Cloud	—	—	—	**nicht neu einsetzen; Dienst endet 22.12.2026**
Cortex/ Convex	Free/usage	Professional $25/ Developer + Usage	Enterprise ab $2.500 Mindestbetrag	**für diesen Zweck nicht sinnvoll**

Qdrant bestätigt den kostenlosen Cluster mit 0,5 vCPU, 1 GB RAM und 4 GB Storage; Paid wird nach CPU, RAM und Storage berechnet. [17] Qdrant nennt für den Free Cluster ungefähr eine Million 768-dimensionale Vektoren als grobe Größenordnung; mit EVA-Textpayload, Indizes und realem Headroom sollte diese Zahl nicht als Kapazitätsgarantie verwendet werden. [18]

Weaviate bietet aktuell einen kostenlosen Tier bis 100.000 Objects und Flex ab 45 US-Dollar pro Monat; das Pricing berücksichtigt unter anderem Vector Dimensions und Storage. Die Preisliste wurde im September 2026 aktualisiert. [19]

Zilliz nennt für 768-dimensionale Vektoren bei Dedicated Clusters grob etwa 65 US-Dollar pro Million Vektoren und Monat für Performance-optimiert, etwa 20 US-Dollar für Capacity-optimiert und rund 7 US-Dollar für Tiered Storage in seiner Cost-Optimization-Dokumentation; tatsächliche Kosten hängen zusätzlich von Storage, Region und Workload ab. [20]

Pinecones gegenwärtige öffentliche Pläne beginnen mit Free, Builder bei 20 US-Dollar, Standard mit mindestens 50 US-Dollar monatlichem Usage und Enterprise mit mindestens 500 US-Dollar. [14]

Bei Cortex/Convex bietet Convex Vector Search bereits im Plattformmodell; Professional kostet derzeit 25 US-Dollar pro Entwickler und Monat, Enterprise hat einen Mindestbetrag von 2.500 US-Dollar, jeweils plus beziehungsweise inklusive unterschiedlicher Usage-Komponenten. [10] Das unterstreicht, warum ein Wechsel von EVAs Index zu Cortex keine reine „Vector-DB-Auswahl“, sondern ein deutlich größerer Plattformwechsel wäre.

Empfohlene Deployment-Varianten für EVA

Deployment	Datenschutz	Ops	HA	Kosten	Empfehlung
Nextcloud + Qdrant auf demselben kleinen Host	sehr gut	niedrig	nein	niedrig	nur kleine/private Instanzen
Nextcloud + separater Qdrant-Container/VM	sehr gut	mittel	optional	niedrig–mittel	**beste Default-Self-host-Variante**
Qdrant 3-Node intern	sehr gut	mittel–hoch	gut	mittel–hoch	größere Organisationen
Qdrant Managed EU	gut bei passender Governance	niedrig	gut	mittel	**beste Managed-Variante**
Weaviate self-host	sehr gut	mittel	möglich	mittel	beste Hybrid-Alternative
Zilliz/Milvus Managed	abhängig von Region/ Governance	niedrig	gut	mittel–hoch	nur bei großem Scale
Pinecone	Cloud-Datenfluss erforderlich	sehr niedrig	managed	mittel–hoch	optionaler Enterprise-Provider
pgvector im Nextcloud-PG	sehr gut	sehr niedrig	folgt PG	niedrig	hervorragender optionaler PG-Adapter

Qdrant bietet Managed-, Hybrid- und Private-Cloud-Modelle und ermöglicht im Managed-Modell unter anderem dedizierte Ressourcen, HA, Backups und Disaster Recovery; Self-host verwendet dieselbe Open-Source-Engine/API. [2] Weaviate kann als Managed Cloud, Docker oder Kubernetes betrieben werden. [21] Milvus unterscheidet Lite, Standalone und Distributed; bei Distributed kommen die verteilten Infrastrukturkomponenten und Kubernetes-typische Betriebsanforderungen hinzu. [22]

Single Points of Failure nach der empfohlenen Migration

Der heutige wichtigste AI-SPOF ist der für Embeddings konfigurierte Ollama-Pfad: Semantic Indexing und normalerweise Query Embedding hängen davon ab; der Code besitzt in bestimmten Situationen zwar lexical fallbacks, diese ersetzen semantisches Retrieval jedoch nicht gleichwertig. Eine `EmbeddingProvider`-Abstraktion reduziert dieses Risiko.

Ein einzelner Qdrant-Node wäre ein **neu eingeführter SPOF**. Deshalb sollte EVA bei kleinen Installationen einen sauber definierten „retrieval unavailable/legacy fallback“-Modus besitzen und bei kritischen Deployments repliziert betrieben werden. Qdrant Cloud bietet Multi-node-/HA-Varianten; der kostenlose Single Node ist ausdrücklich nicht HA. [13]

Die Nextcloud-Datenbank bleibt weiterhin die wichtigste Source-of-Truth-Abhängigkeit. Das ist kein EVA-spezifischer Architekturfehler, sondern eine Konsequenz der App-Integration. Der zusätzliche Vector Store sollte gerade deshalb **rekonstruierbar** und nicht alleinige Datenautorität werden.

`chats.json` bleibt ohne weitere Migration ein per-user Failure-/Contention-Domain. Die Verschlüsselung reduziert Vertraulichkeitsrisiken, aber nicht die strukturelle Rewrite-/Lock-Problematik.

Abschließende technische Empfehlung: EVA sollte nicht zu einer Sammlung unabhängiger AI-Microservices umgebaut werden. Die Nextcloud-nahe Architektur ist ein Vorteil. Der sinnvolle Schnitt liegt genau dort, wo spezialisierte Infrastruktur einen klaren algorithmischen Vorteil bringt: **Vector/Hybrid Retrieval** und optional **Embedding Inference**. SQL sollte Autorität für Documents, Chunks und Anwendungszustand bleiben; Qdrant sollte einen rekonstruierbaren Search Index darstellen; Provider Profiles sollten Chat und Embeddings über getrennte Interfaces abstrahieren; Chats sollten mittelfristig normalisiert werden. Damit bleibt die Architektur für kleine Self-hosted Nextcloud-Instanzen betreibbar, beseitigt aber den O(N·d)-Vollscan, bevor dieser zum fundamentalen Skalierungsdeckel von EVA wird.

Einschränkung der Analyse: Es bestand, wie vorgegeben, kein Zugriff auf eine laufende Nextcloud-/EVA-Instanz. Aussagen zu tatsächlicher p95/p99-Latenz, RAM-Verbrauch, Chunk-Zahlen, DB-Größe, Provider-Latenz und realer Retrieval-Qualität können deshalb nicht gemessen werden. Die Performance- und Aufwandsschätzungen sind Architekturprognosen aus Repository-Code und offiziellen Produktdokumentationen; der vorgeschlagene Shadow-Benchmark ist gerade dazu gedacht, diese Annahmen vor einem Produktions-Cutover empirisch zu bestätigen.

---

1 11 Hybrid Queries - Qdrant
https://qdrant.tech/documentation/hybrid-queries/?utm_source=chatgpt.com

2 13 17 Pricing for Cloud and Vector Database Solutions Qdrant
https://qdrant.tech/pricing/?utm_source=chatgpt.com

3 Hybrid search | Weaviate Documentation
https://docs.weaviate.io/weaviate/concepts/search/hybrid-search?utm_source=chatgpt.com

4 Milvus Architecture Overview | Milvus Documentation
https://milvus.io/docs/architecture_overview.md?utm_source=chatgpt.com

5 14 Pricing | Pinecone
https://www.pinecone.io/pricing/?utm_source=chatgpt.com

6 Chroma Cloud wind-down timeline | Chroma
https://www.trychroma.com/cloud-wind-down?utm_source=chatgpt.com

7 Client - Chroma Docs
https://docs.trychroma.com/reference/python?utm_source=chatgpt.com

8 pgvector/README.md at master · pgvector/pgvector · GitHub
https://github.com/pgvector/pgvector/blob/master/README.md?plain=1&utm_source=chatgpt.com

9 System requirements — Nextcloud 35 Administration Manual
https://docs.nextcloud.com/server/stable/admin_manual/installation/system_requirements.html?utm_source=chatgpt.com

10 Choose Your Plan
https://www.convex.dev/pricing?sid=bd18d511-9507-41b9-8b08-9d01c114671d&utm_source=chatgpt.com

12 Qdrant Cloud: Scalable Managed Cloud Services
https://qdrant.tech/cloud/?utm_source=chatgpt.com

15 Introduction - Chroma Docs
https://docs.trychroma.com/?utm_source=chatgpt.com

16 Inference - Qdrant
https://qdrant.tech/documentation/cloud/inference/?utm_source=chatgpt.com

18 Create a Cluster - Qdrant
https://qdrant.tech/documentation/cloud/create-cluster/?q=high&utm_source=chatgpt.com

19 Vector Database Pricing | Weaviate
https://weaviate.io/pricing?utm_source=chatgpt.com

20 Cost Optimization | Cloud | Zilliz Cloud Developer Hub
https://docs.zilliz.com/docs/cost-optimization?utm_source=chatgpt.com

21 Deploying Weaviate | Weaviate Documentation
https://docs.weaviate.io/deploy?utm_source=chatgpt.com

22 Overview of Milvus Deployment Options | Milvus Documentation
https://milvus.io/docs/install-overview.md?utm_source=chatgpt.com